

Guia Completo: Protegendo Credenciais no Spring Boot

Este guia assume que você nunca fez isso antes. Siga a ordem exata.

PARTE 1: Trocar a senha do banco AGORA (urgente)

Antes de qualquer coisa técnica, troque a senha do MySQL. Ela já apareceu em texto puro numa conversa e provavelmente já está salva no histórico do Git.

1. Acesse o servidor MySQL (via terminal, DBeaver, MySQL Workbench, etc).
2. Execute:

```
ALTER USER 'junior'@'%' IDENTIFIED BY 'NovaSenhaForteAqui123!';  
FLUSH PRIVILEGES;
```

3. Guarde essa nova senha em um lugar seguro (gerenciador de senhas, não em bloco de notas).

Se você não tem acesso para trocar a senha, peça para o responsável pelo banco (DBA/infra) fazer isso ainda hoje.

PARTE 2: Editar o `application.properties`

1. Abra o arquivo `src/main/resources/application.properties` no seu projeto.
2. Substitua as linhas de usuário e senha por isso:

```
spring.datasource.username=${DB_USERNAME}  
spring.datasource.password=${DB_PASSWORD}
```

3. Salve o arquivo.

Isso diz ao Spring: "pegue o valor da variável de ambiente `DB_USERNAME` e `DB_PASSWORD` quando o sistema iniciar".

PARTE 3: Criar o arquivo `.env` (onde a senha real vai ficar)

1. Na **raiz do projeto** (mesma pasta onde está o `pom.xml`), crie um arquivo chamado exatamente `.env` (sem nome antes do ponto).
 - No Windows, se o Explorer não deixar criar arquivo só com ponto, abra o VS Code/IntelliJ, clique com botão direito na pasta *New File* digite `.env`.
2. Abra esse arquivo e escreva:

```
DB_USERNAME=junior
DB_PASSWORD=NovaSenhaForteAqui123!
```

3. Salve.

Esse arquivo **nunca** vai para o Git. É só local, na sua máquina.

PARTE 4: Fazer o Spring Boot ler o `.env` automaticamente

Adicione esta dependência no seu `pom.xml`, dentro da tag `<dependencies>`:

```
<dependency>
  <groupId>me.paulschwarz</groupId>
  <artifactId>spring-dotenv</artifactId>
  <version>4.0.0</version>
</dependency>
```

Depois, no terminal, dentro da pasta do projeto, rode:

```
mvn clean install
```

Isso baixa a biblioteca e recompila o projeto. Com ela instalada, o Spring Boot lê o `.env` sozinho, sem configuração extra.

PARTE 4.1: Alternativa mais confiável — variáveis direto na IDE

Em alguns casos (principalmente no IntelliJ), rodar a aplicação pelo botão **Run** da IDE não carrega o `.env` corretamente, mesmo com o `spring-dotenv` instalado. Se isso acontecer, configure as variáveis direto na IDE:

1. Vá em **Run Edit Configurations...**
2. Selecione a configuração da sua aplicação (ex: `SaacSecurityApplication`)
3. Procure o campo **Environment variables**
4. Clique no ícone de pasta/editar e adicione:

```
DB_USERNAME=junior
DB_PASSWORD=sua_senha_nova_aqui
```

5. Clique **Apply OK**
6. Rode novamente com o botão **Run**

Essa opção funciona de forma mais garantida porque a IDE injeta as variáveis diretamente no processo da aplicação, sem depender da leitura do arquivo `.env`.

PARTE 5: Garantir que o `.env` nunca seja commitado

1. Abra (ou crie) o arquivo `.gitignore` na raiz do projeto.
2. Adicione estas linhas no final:

```
.env
*.env
```

3. Salve.
4. Verifique se o Git já não está rastreando o `.env` (caso você já tenha dado `git add` nele antes):

```
git status
```

Se aparecer `.env` na lista de "staged" ou "modified", rode:

```
git rm --cached .env
```

Isso remove do controle do Git **sem apagar o arquivo da sua máquina**.

PARTE 6: Testar se funcionou

1. Rode a aplicação normalmente pela sua IDE (botão de Run) ou via terminal:

```
mvn spring-boot:run
```

2. Se a aplicação subir sem erro de conexão com o banco, funcionou.

3. Se der erro do tipo `Could not resolve placeholder 'DB_USERNAME'`, significa que o `.env` não foi encontrado — confira se ele está exatamente na raiz do projeto e se o nome está certo (`.env`, sem extensão depois).

PARTE 7: Limpar a senha que já está no histórico do Git

Esta parte é a mais delicada. Como a senha antiga já foi commitada, ela continua visível no histórico mesmo depois de você remover do arquivo atual. Como você já trocou a senha na Parte 1, o risco já está neutralizado — mas é boa prática limpar o histórico também, principalmente se o repositório for compartilhado ou público (GitHub, GitLab).

Se o repositório é privado e só você usa:

Pode deixar para depois, não é urgente, já que a senha antiga foi trocada e não funciona mais.

Se o repositório é público ou tem outras pessoas com acesso:

Use a ferramenta **BFG Repo-Cleaner** (mais simples que `git filter-repo`):

1. Baixe o BFG: <https://rtyley.github.io/bfg-repo-cleaner/> (é um `.jar`, precisa ter Java instalado).

2. Faça um clone limpo (espelho) do seu repositório:

```
git clone --mirror https://github.com/seu-usuario/seu-repo.git
```

3. Entre na pasta clonada e rode:

```
java -jar bfg.jar --replace-text senhas.txt seu-repo.git
```

Onde `senhas.txt` é um arquivo de texto simples contendo:

```
100376
```

4. Entre na pasta `seu-repo.git` e force o push:

```
cd seu-repo.git  
git reflog expire --expire=now --all && git gc --prune=now --aggressive  
git push --force
```

Avisar qualquer pessoa que tenha clonado o repositório antes, pois ela precisará clonar de novo após essa limpeza (o histórico muda).

PARTE 8: Em produção (quando for colocar no servidor de verdade)

Quando for publicar a aplicação (não só rodar na sua máquina), **não use arquivo `.env`**. Defina as variáveis diretamente no ambiente:

Se usar Docker:

```
docker run -e DB_USERNAME=junior -e DB_PASSWORD=NovaSenhaForteAqui123! sua-  
imagem
```

Se usar docker-compose.yml:

```
services:  
  app:  
    image: sua-imagem
```

environment:

- DB_USERNAME=junior
- DB_PASSWORD=NovaSenhaForteAqui123!

Se usar um servidor Linux direto (systemd): Edite o arquivo de serviço (ex: `/etc/systemd/system/saac.service`) e adicione:

```
[Service]
Environment="DB_USERNAME=junior"
Environment="DB_PASSWORD=NovaSenhaForteAqui123!"
```

Checklist final

- ☐ Senha do banco trocada
- ☐ `application.properties` usando `${DB_USERNAME}` e `${DB_PASSWORD}`
- ☐ Arquivo `.env` criado na raiz com os valores reais
- ☐ Dependência `spring-dotenv` adicionada no `pom.xml`
- ☐ `.gitignore` atualizado com `.env`
- ☐ `.env` removido do rastreamento do Git (`git rm --cached`)
- ☐ Aplicação testada e rodando normalmente
- ☐ Histórico do Git limpo (se repositório público/compartilhado)